課堂筆記整理｜2026 年 3 月 27 日

AI Coding 框架比較：Superpowers、GSD、gstack 與 Auto Research

主題聚焦：AI 寫碼可信度、流程治理、上下文工程、角色治理，以及未來持續優化方向

這段影片探討一個很實際的問題：今天 AI 寫 code 最大痛點，已經不是「**寫不寫得出**」，而是「**寫完之後可不可靠**」。測試夠唔夠？安全性有冇漏洞？架構有冇崩？長遠維護會唔會更麻煩？影片透過比較三個熱門框架——Superpowers、GSD、gstack，再延伸到 Auto Research，分析目前 AI coding 工具到底各自解決緊甚麼問題，以及最大缺口在哪裡。

> **核心觀點：**
> 這三個框架表面似乎互相競爭，其實分別在解決**流程**、**環境**、**視角**三種不同問題；而真正仍未被完整解決的，是**構建階段的可靠性與持續優化**。

一、Superpowers：用流程紀律來保證代碼質量

Superpowers 的出發點很直接：AI 不是能力不夠，而是**太散漫**。你叫它加一個功能，它可能真係會幫你加完，但可能冇測試、冇按原本設計做、質量忽高忽低。這種情況就像團隊入面那些寫 code 很快、但從來不寫測試的初級工程師。

Superpowers 的方法是把開發變成一條**嚴格流水線**：先做結構化需求對話，再生成詳細實施計劃，再強制走 TDD。換句話說，它的重點不在「叫 AI 更自由」，而是透過流程紀律去令輸出更穩定。

二、GSD：用上下文工程來對抗 Context Rot

GSD 對準的是另一個問題：**上下文腐爛**。當你同 Claude 聊太耐、上下文窗口用得太深，模型會開始偷懶、遺忘、甚至胡說八道。GSD 的解法非常乾脆：**徹底隔離上下文**。

GSD 的核心手法：

1. 把工作拆成原子任務
2. 每個任務都開全新 Claude session
3. 主會話只做調度，不做重活

4. 所有狀態與進度都寫入磁碟文本，方便重讀

它的核心假設是：AI 質量很大程度取決於上下文有幾乾淨。**環境乾淨，輸出就穩定**。

三、gstack：用角色與流程治理來提升判斷質量

gstack 的切入點又唔同。它唔係先管你點樣寫 code，也唔係先管 context 淨唔淨，而係先管：**邊個角色應該做邊種判斷**。所以它建立咗一套角色治理系統，有產品、規劃、工程審查、QA、ship 等多個角色，令 Claude 在不同階段以不同視角去思考。

表面看，gstack 好似只是幫 Claude 起了 15 個名字，但影片拆解後指出，gstack 其實至少在做五件事：**角色聚焦**、**數據編排**、**質量管控**、**決策偏好注入**，以及**認知負載適配**。

四、gstack 最值得注意的缺口：構建階段真空

影片最有洞察力的地方，是指出 gstack 流程雖然完整涵蓋了「思考、規劃、構建、審查、測試、發布」，但實際上在**構建階段**幾乎冇額外治理。規劃過後，Claude Code 便回到 default mode，沒有 TDD、沒有子代理、沒有增量審查，要到你手動再叫 review 時，約束才重新出現。

而偏偏構建階段，正正就是 Superpowers 與 GSD 最擅長的地方。所以影片提出一個自然想法：能否把 gstack 的規劃與審查，和 Superpowers 的構建能力結合？理論上可以互補，但實際卡住在一個技術問題——Superpowers 在構建時會彈出互動式提問，令 Claude Code 卡住，其他 agent 也無法再接手。

五、Auto Research：一條新的方向

影片接著引入 Andrej Karpathy 的 Auto Research 模式，作為一個可能突破。其核心非常簡單：AI 修改代碼 → 跑固定時間測試 → 量度單一指標 → 指標改善就保留，否則就重置，再進入下一輪。這是一個**單代理緊密循環**，不依賴多個 agent 互相溝通，因此不容易被互動卡死。

若把這個思路搬進軟件工程，就可以想像一個 optimize skill 自動改代碼、跑測試、量度覆蓋率、Lighthouse 分數、bundle size，然後根據數字決定保留或丟棄。這樣不只是填補「構建階段」的空白，而是讓 gstack 在發佈之後多一層**無人值守的持續優化**能力。

六、總結：三者互補，未來在於循環式優化

影片最後的總結很清楚：**Superpowers 約束流程，GSD 約束環境，gstack 約束視角**。三者其實並非真正競爭，而是互補。gstack 最強的是產品思維與質量管控，但最大缺口就在構建階段；Superpowers 與 GSD 剛好可以補這塊，但跨框架整合仍有技術瓶頸。

而真正值得進一步探索的新方向，就是 **Auto Research 那種以單一可量化指標驅動、反覆實驗、反覆保留更好版本** 的循環機制。這意味著未來 AI coding 工具的重點，可能不再只是「幫你生成」，而是「幫你持續改進」。

> **帶走重點：**
> 今天 AI coding 問題的核心，不再是能不能寫，而是怎樣令它寫得**可靠**、**可維護**、**可持續優化**。而這件事，單靠 prompt 已經不夠，必須上升到流程治理與系統設計層面。